# AEGIS - SECURE P2P FILE SHARING SYSTEM


**Submitted By**

Project Leader: Muhammad Daniyal | 24K-2048

Team Member: Hassan Arif | 24K-2037


Computer Networks — CS3001 | Spring 2026


**FAST School of Computing**

**National University of Computer & Emerging Sciences**

# 1. Motivation

*The growth of digital data and the increasing need for secure, decentralized communication has made peer-to-peer (P2P) systems a subject of critical importance in modern networking. Centralized file-sharing services, while convenient, introduce single points of failure, privacy vulnerabilities, and bandwidth bottlenecks that are fundamentally at odds with the open nature of the Internet. This project is motivated by the desire to build a functional, real-world P2P file-sharing system from the ground up — one that demonstrates the core principles taught in this course, namely TCP/IP socket programming, UDP-based service discovery, and cryptographically secure data transmission.*

# 2. Overview

Aegis is a fully decentralized, serverless application written in Python and Flask. Each instance of the application acts as both a server and a client simultaneously. Nodes advertise their presence on the local network via UDP broadcast and maintain an up-to-date registry of active peers without any central coordinator.

File transfers are executed over raw TCP sockets with AES-256 GCM authenticated encryption applied chunk-by-chunk. Large files are split into 5 MB work units and downloaded in parallel from multiple peers simultaneously (swarm downloading), maximizing throughput and resilience. Every downloaded file is verified against a SHA-256 hash announced by the seeding peer, and peer-discovery broadcasts are HMAC-signed to prevent spoofing. A modern, dark-themed web UI (served by Flask) provides full control over the system from any browser.

## 2.1 Significance of the Project

### 2.1.1 Academic Value

This project directly applies concepts from multiple weeks of the Computer Networks(CS3001) curriculum: TCP socket programming, UDP broadcast, application-layer protocol design, concurrent threading, and network security principles. It serves as a cohesive demonstration that these concepts work together in a real, working system.

### 2.1.2 Practical Importance

P2P architectures power some of the most significant file-distribution systems ever built (BitTorrent, IPFS, Napster). Understanding their implementation at the socket level is essential knowledge for any network engineer. Our system replicates the core mechanisms of these protocols in a clean, readable codebase.

### 2.1.3 Security Relevance

By integrating AES-256 GCM authenticated encryption, HMAC-signed peer discovery, and SHA-256 integrity verification, the project demonstrates that security need not be an afterthought in networking applications. Every byte transferred is encrypted and tamper-evident, every completed file is verified, and discovery messages are authenticated before being trusted — a requirement in any production-grade system.

### 2.1.4 Scalability and Resilience

The swarm-download mechanism means that the system becomes more capable as more peers join, rather than degrading under load. If one peer drops mid-transfer, workers simply pick up from the remaining available peers, providing fault tolerance with no additional coordination overhead.

## 2.2 Description of the Project

The system consists of three main Python modules and one HTML frontend:

- peer_node.py — The core networking engine. Manages UDP discovery, TCP server, swarm download orchestration, AES encryption/decryption, SHA-256 hashing, and the transfer log.
- crypto_utils.py — Cryptographic utilities. Derives a 32-byte AES-256 key from a shared secret using SHA-256 and provides encrypt_chunk() / decrypt_chunk() functions using GCM mode with random nonces and built-in authentication tags.
- app.py — Flask web server and REST API layer. Exposes all peer operations (file listing, transfer requests, peer discovery, upload/delete) via JSON endpoints, with async job tracking for downloads. A browser heartbeat keeps the node alive and shuts it down automatically if the UI tab is closed.
- index.html — Single-page dark-themed web UI. Communicates with the Flask backend via REST calls to provide real-time peer monitoring, file management, search, and transfer progress.

The system operates over two network channels: a UDP broadcast channel (port 54321) for peer advertisement and discovery, and per-peer TCP sockets for file listing and actual file transfer. The Flask web server runs on a separate port and is accessible only from localhost, serving as the control plane for the node's UI.

## 2.3 Background of the Project

### 2.3.1 Peer-to-Peer Networking

Peer-to-peer networks are distributed systems in which each participant (peer) acts simultaneously as both a client and a server. Unlike the client-server model, there is no dedicated central server; resources are shared

directly between peers. This architecture was popularized by Napster (1999), Gnutella (2000), and BitTorrent (2001) [1].

### 2.3.2 Socket Programming

The project uses Python's socket library to create TCP and UDP sockets. TCP provides reliable, ordered, connection-oriented communication suitable for file transfer, while UDP provides lightweight, connectionless datagrams ideal for broadcast discovery [2].

### 2.3.3 AES-256 GCM Encryption

The Advanced Encryption Standard (AES) with 256-bit keys in Galois/Counter Mode (GCM) is used for all chunk encryption. A random 12-byte nonce is generated per chunk, and encryption produces a 16-byte authentication tag alongside the ciphertext; both are prepended to the chunk before transmission. GCM provides authenticated encryption, so any tampering with a chunk in transit is detected at decryption time rather than silently passed through, as could happen with CBC. The shared key is derived from a passphrase using SHA-256, implemented via the PyCryptodome library [3].

### 2.3.4 SHA-256 Integrity Verification

After downloading, the receiver computes the SHA-256 hash of the complete file and compares it to the hash announced by the seeding peer in the LIST response. This cryptographically guarantees that the received file is identical to the original and has not been corrupted or tampered with in transit [4].

### 2.3.5 Swarm Downloading

Inspired by BitTorrent's piece-based downloading, the system divides files into 5 MB work orders distributed across a thread-safe queue. Multiple worker threads — one per available peer — consume work orders concurrently, writing their chunks directly to the correct offset in a pre-allocated file. This allows download speed to scale linearly with the number of seeding peers [5].

## 2.4 Project Category

This is a Product-Based project. It produces a fully functional, deployable application that can be used for real file sharing within a local area network (LAN). The project demonstrates applied knowledge of computer networking, cryptography, concurrent programming, and web development.

# 3. Features / Scope / Modules

### 3.1 Automatic Peer Discovery (UDP Broadcast)

Every peer broadcasts its identity (name, TCP port, Flask port) every 5 seconds over UDP port 54321. Peers listen on the same port and update their known-peers registry accordingly. Stale peers (not seen in 15 seconds) are automatically evicted. This zero-configuration discovery means peers find each other with no manual IP entry.

### 3.2 AES-256 GCM Encrypted File Transfer

Every 4 KB chunk of file data is individually encrypted with AES-256 GCM before transmission and decrypted upon receipt. A fresh random nonce per chunk ensures semantic security, and the built-in authentication tag means corrupted or tampered chunks fail decryption instead of being silently accepted. This makes passive eavesdropping on the TCP stream useless — an adversary capturing traffic sees only ciphertext.

### 3.3 Multi-Source Swarm Downloading

When multiple peers share the same file, the downloader splits the file into 5 MB work orders and assigns them to concurrent worker threads, one per peer. Failed chunks are re-queued automatically. The result is download speeds that scale with peer count, and automatic failover if a peer disconnects mid-transfer.

### 3.4 SHA-256 File Integrity Verification

Upon completing a download, the system computes the SHA-256 hash of the received file and compares it against the hash provided by the seeding peer. A green checkmark (verified) or red warning (mismatch) is displayed in the transfer log and the job status API.

### 3.5 Real-Time Browser-Based UI

The Flask server exposes a REST API consumed by a single-page HTML/JS application. The UI provides: live peer list with ping status, shared and received file management, cross-peer file search, chunked file upload, download with real-time progress bar and speed indicator, and a reverse-chronological transfer log.

### 3.6 Parallel Search Across All Peers

The /api/search endpoint launches one thread per known peer, queries each peer's file list concurrently, and aggregates results matching the search query. Results are returned to the UI within the TCP timeout window regardless of peer count.

### 3.7 Hash Cache

To avoid redundant SHA-256 computation when serving the LIST command to multiple requesters, peer_node maintains a hash cache keyed by file path and modification timestamp. The cache is invalidated automatically when a file is modified or deleted.

### 3.8 Network and Transfer Hardening

Discovery broadcasts are signed with HMAC-SHA256 using the shared secret, so a peer drops any announcement with an invalid signature instead of trusting it blindly. File requests are protected against path traversal: filenames are sanitized with secure_filename() and downloaded files are confirmed to resolve inside the received_files directory before being written. List responses and chunk sizes are also capped, so a misbehaving or malicious peer cannot force a node to allocate unbounded memory.

## 4. Project Planning

The project was completed over a four-week sprint, with responsibility divided between the two team members as outlined below.

| Week | Tasks | Lead | Status |
|---|---|---|---|
| Week 1 | UDP discovery, TCP server, PING/LIST/GET protocol design | 24K-2048 | Complete |
| Week 2 | AES-256 GCM encryption, SHA-256 verification, chunk protocol | 24K-2037 | Complete |
| Week 3 | Flask REST API, swarm download, async job tracker | 24K-2048 | Complete |
| Week 4 | Web UI, integration testing, bug fixes, report | Both | Complete |

Table 1. Project Planning and Task Allocation

## 5. Project Feasibility

### 5.1 Technical Feasibility

The project relies exclusively on Python standard library modules (socket, threading, hashlib, json, struct, queue, os) plus two well-established packages: Flask for the web server and PyCryptodome for AES encryption. All are cross-platform and available via pip. Python's threading model is sufficient for the concurrency requirements given that the bottleneck is network I/O, not CPU. The project is fully technically feasible and has been demonstrated to work.

### 5.2 Economic Feasibility

The project has zero cost. All software used is open-source (Python, Flask, PyCryptodome). No cloud infrastructure or paid services are required; the system operates entirely within a local network. Hardware requirements are any two or more computers on the same LAN — standard lab equipment is sufficient.

### 5.3 Schedule Feasibility

The project was completed within the four-week semester timeline. The modular architecture (separate discovery, transfer, encryption, and UI layers) allowed parallel development by the two team members with minimal merge conflicts. The use of Python's built-in libraries reduced dependency management overhead significantly.

## 6. Hardware and Software Requirements

### 6.1 Hardware

- 2 or more computers connected on the same Local Area Network (LAN), or a single machine running multiple peer instances (for testing)
- Minimum: 512 MB RAM, any modern CPU
- Network: any Ethernet or Wi-Fi connection supporting UDP broadcast

### 6.2 Software

- Python 3.10 or later
- Flask 3.x (pip install flask)
- PyCryptodome 3.x (pip install pycryptodome)
- A modern web browser (Chrome, Firefox, Edge) for the UI

Operating System: Windows 10/11, macOS 12+, or Ubuntu 20.04+ (all tested)

## 7. Diagrammatic Representation of the Overall System

The figures below illustrate the network topology of Aegis. Each peer runs a Flask web application backed by a PeerNode engine that manages UDP discovery, local file storage, and an encrypted TCP server. File transfers are executed in parallel across all available seeding peers, with SHA-256 verification performed on the fully assembled file upon completion.


Figure 1. P2P Network Architecture

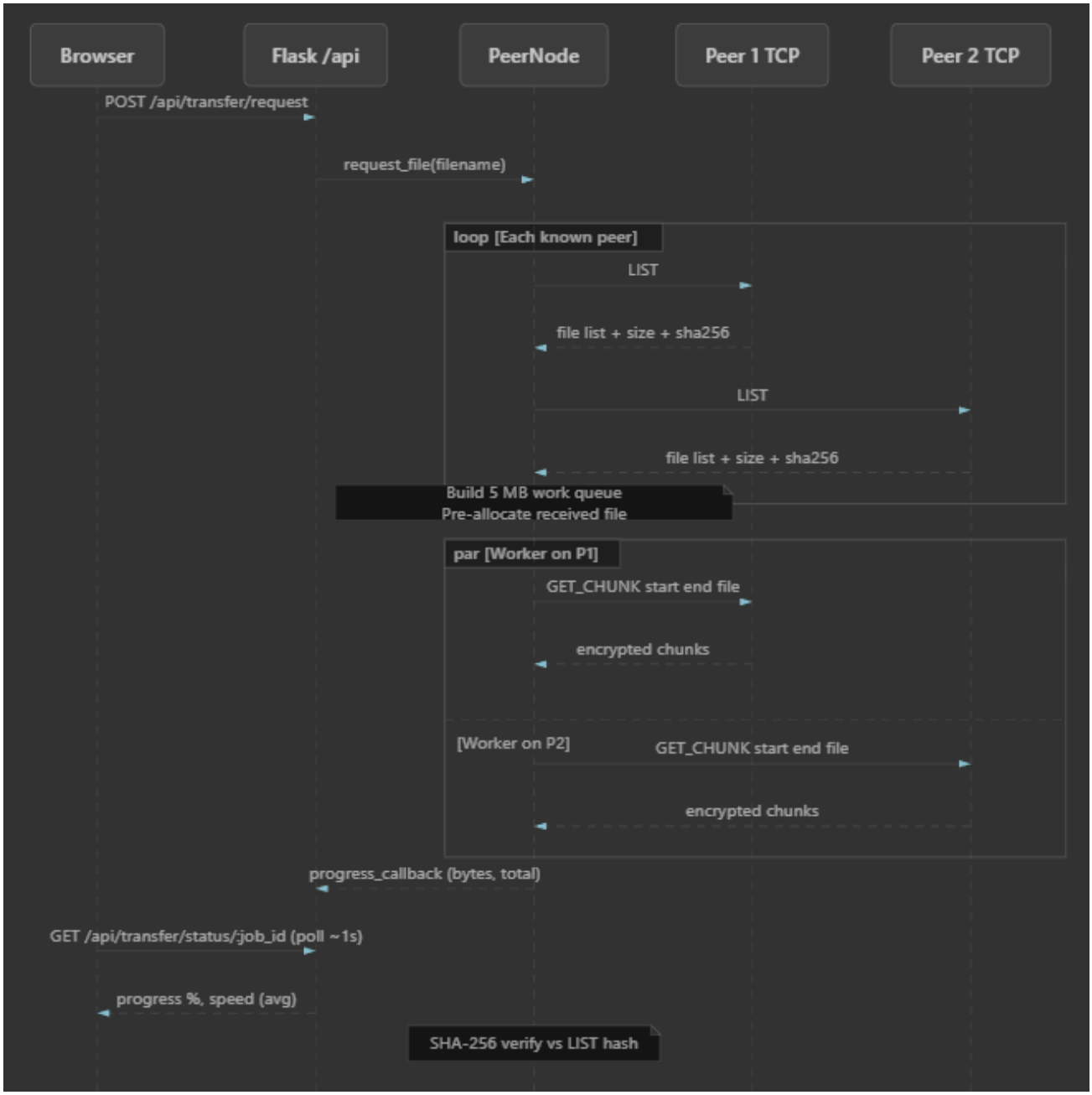


Figure 2. File Transfer Sequence Diagram

## 8. Working Demo — Screenshots

The following screenshots demonstrate the complete functionality of Aegis running on a local network with multiple peer instances.

### 8.1 File Management Dashboard

The main dashboard shows the node's shared files (available to other peers) and received files (downloaded from peers). Files can be uploaded from local disk in chunked form and deleted from either directory.

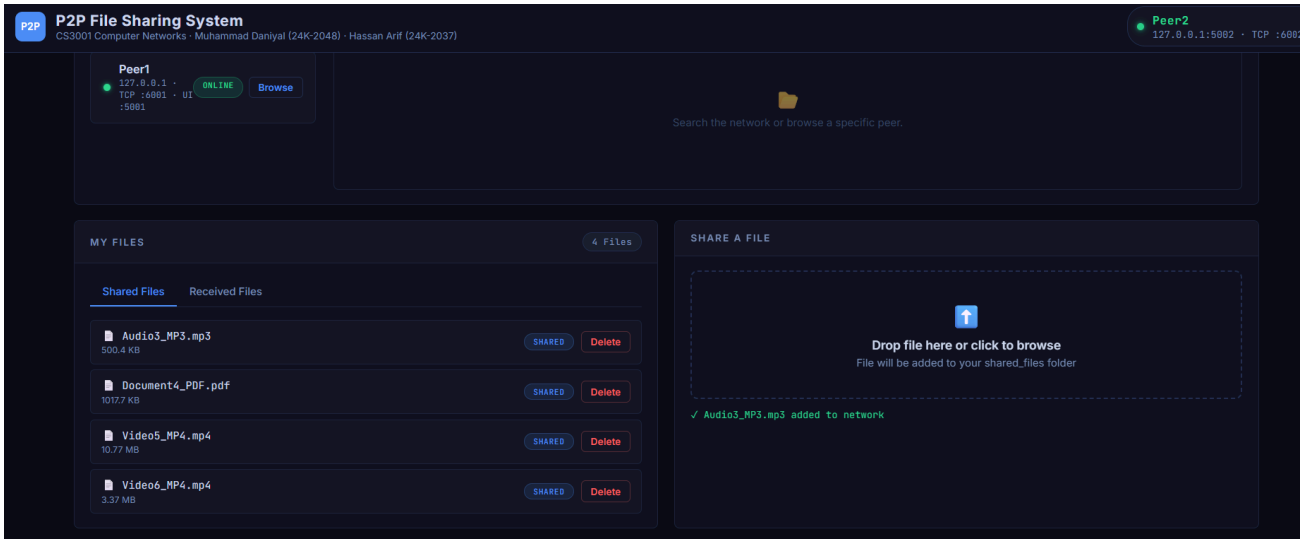


Figure 3. My Files Dashboard — Shared and Received Files Management

### 8.2 Live Peer Discovery

The Peers panel displays all nodes currently active on the network. Peers are discovered automatically via UDP broadcast, so no manual configuration is required. Each entry shows the peer name, IP address, TCP port, Flask port, and online status. A ping test can be initiated from the UI to verify connectivity.

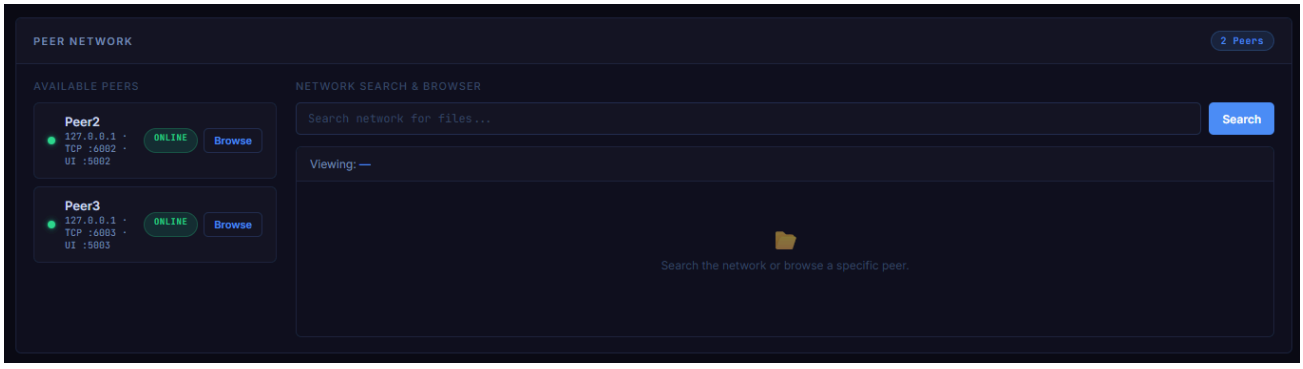


Figure 4. Active Peers Panel — Auto-Discovered via UDP Broadcast

## 8.3 Real-Time File Transfer

When a download is initiated, the system checks all peers for the file, splits it into 5 MB work orders, and assigns them to concurrent download workers. The progress bar updates in real time, showing completion percentage and current download speed. Multiple peers contribute simultaneously in the swarm download mode.

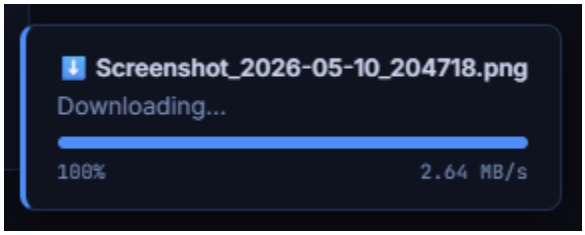


Figure 5. Active Swarm Download — Real-Time Progress and Speed Indicator

## 8.4 Global File Search

The search function queries all active peers simultaneously in parallel threads. Results are aggregated and displayed with the owning peer's name, filename, and file size. Any result can be downloaded with a single click, which initiates the swarm download process described above.

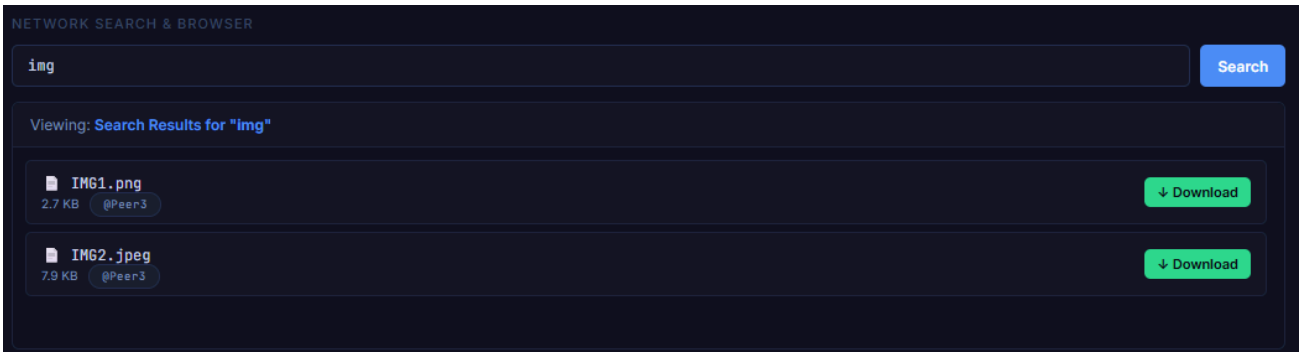


Figure 6. Cross-Peer File Search — Results Aggregated from All Active Nodes

## 8.5 Transfer Log with Integrity Status

Every completed transfer (sent or received) is recorded in the transfer log with timestamp, direction, filename, source peer, size, and SHA-256 verification status. A 'VERIFIED' badge indicates that the SHA-256 hash of the received file matches the hash announced by the seeding peer, confirming end-to-end integrity.

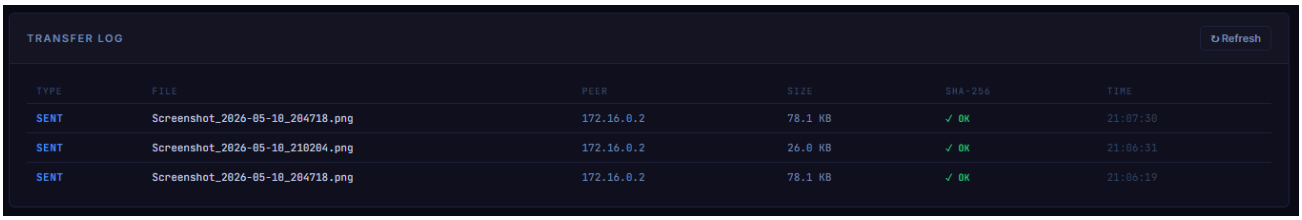


Figure 7. Transfer Log — All Transfers with SHA-256 Verification Status

## 9. References

[1] B. Cohen, "Incentives Build Robustness in BitTorrent," Workshop on Economics of Peer-to-Peer Systems, 2003.

[2] W. R. Stevens, B. Fenner, and A. M. Rudoff, UNIX Network Programming, Vol. 1, 3rd ed., Addison-Wesley, 2004.

[3] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," FIPS Publication 197, 2001.

[4] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," FIPS Publication 180-4, 2015.

[5] Python Software Foundation, socket - Low-level networking interface. Python 3.12 Documentation. Available: https://docs.python.org/3/library/socket.html

[6] Pallets Projects, Flask - A lightweight WSGI web application framework. Available: https://flask.palletsprojects.com/

[7] Helder Eijs, PyCryptodome - A self-contained Python package of cryptographic primitives. Available: https://www.pycryptodome.org/